

MANUAL DE INTEGRACAO COM INTELIGENCIA ARTIFICIAL

Appointments API — Prompt + Guia Passo a Passo

appointments-api-five.vercel.app

INDICE

1. Prompt completo para IA (copiar e colar)
2. Passo 1 — Criar um novo negocio na API
3. Passo 2 — Configurar variaveis de ambiente
4. Passo 3 — Setup de admin (primeira vez)
5. Passo 4 — Configurar funcionarios, servicos e horarios
6. Passo 5 — Deploy em producao (Vercel)
7. Resolucao de problemas frequentes

1. Prompt completo para IA

Copia e cola o prompt abaixo diretamente na IA logo após criar o website. Substitui os valores entre [COLCHETES] pelos teus dados reais antes de colar.

```
Integra a Appointments API (https://appointments-api-five.vercel.app) neste projeto.
Ja tenho um negocio registrado. A minha API key e: [MINHA_API_KEY]

== VARIÁVEIS DE AMBIENTE ==
Next.js: NEXT_PUBLIC_API_URL=https://appointments-api-five.vercel.app
NEXT_PUBLIC_API_KEY=[MINHA_API_KEY]
Vite/React: VITE_API_URL=https://appointments-api-five.vercel.app
VITE_API_KEY=[MINHA_API_KEY]

== HELPER DE FETCH (TypeScript) ==
async function apiFetch(endpoint, options = {}) {
  const res = await fetch(`${API_URL}${endpoint}`, {
    ...options,
    headers: { 'Content-Type': 'application/json',
      'x-api-key': API_KEY, ...options.headers }
  });
  return res.json();
}
// Admin: adiciona header 'Authorization': `Bearer ${localStorage.getItem('admin_token')}``
```

```
== ENDPOINTS PUBLICOS (x-api-key obrigatorio) ==
GET /api/empleados
-> { empleados: [{ id, nombre, email, telefono }] }

GET /api/servicios?empleadoId={id}
-> { servicios: [{ id, nombre, duracion, precio }] }

POST /api/disponibilidad-multiple
body: { fechas: string[], servicioId: number, empleadoId: number }
-> { resultados: [{ fecha, disponible, huecos: string[], horasOcupadas }] }

POST /api/citas
body: { nombreCliente, emailCliente, fecha: 'YYYY-MM-DDTHH:MM:00',
servicioId, empleadoId }
-> { cita } | { error: string } [envia email automatico ao cliente]
```

```
== WIZARD DE RESERVAS (5 passos) ==
Passo 1: GET /api/empleados -> lista, utilizador seleciona um
Passo 2: GET /api/servicios?empleadoId=X -> servicios do especialista
Passo 3: POST /api/disponibilidad-multiple com proximos 14 dias
-> filtrar disponible=true -> mostrar dias
Passo 4: Mostrar huecos[] como botoes, horasOcupadas[] tachados/desativados
Passo 5: Formulario nome+email -> POST /api/citas -> sucesso ou mensagem de erro
```

```
Utilitarios:
formatarFecha(date: Date) -> 'YYYY-MM-DD'
getProximasDosSemanas() -> array de 14 datas a partir de amanha
DIAS_PT = ['Domingo', 'Segunda', 'Terca', 'Quarta', 'Quinta', 'Sexta', 'Sabado']
MESES_PT = ['Jan', 'Fev', 'Mar', 'Abr', 'Mai', 'Jun', 'Jul', 'Ago', 'Set', 'Out', 'Nov', 'Dez']
```

== SISTEMA ADMIN ==

Rotas a criar: /admin/setup /admin (login) /admin/dashboard

Fluxo de autenticacao:

GET /api/admin/check?apiKey={KEY} -> { configurado: boolean }
POST /api/admin/setup body: { apiKey, password (min 8 chars) }
POST /api/admin/login body: { apiKey, password } -> { token, negocio }
localStorage.setItem('admin_token', token)

Cada pedido admin: header 'Authorization': 'Bearer ' + token

Tab Marcacoes:

GET /api/admin/citas?skip=0&take;=10&fecha;=YYYY-MM-DD&empleadoId;=X
PATCH /api/admin/citas?id=X body: { estado: 'confirmada'|'cancelada' }
Implementar: calendario mini, filtro por funcionario, paginacao

Tab Funcionarios:

GET/POST/PATCH/DELETE /api/admin/empleados body:{nombre,email?,telefono?}
Implementar: lista com edicao inline, formulario de adicionar

Tab Servicios:

GET/POST/DELETE /api/admin/servicios body:{nombre,duracion,precio?,empleadoId}
Nao eliminar se tiver marcacoes associadas – mostrar mensagem de erro

Tab Horarios:

GET/POST/DELETE /api/admin/horarios
body:{diaSemana:'lunes'|'martes'|'miercoles'|'jueves'|'viernes'|'sabado'|'domingo',
horaInicio:'HH:MM', horaFin:'HH:MM', empleadoId}
ATENCAO: diaSemana em ESPANHOL ('lunes'...'domingo')

Tab Pausas:

GET/POST/DELETE /api/admin/pausas (mesma estrutura de horarios)
A pausa deve estar dentro do horario do dia selecionado

Tab Dias Bloqueados:

GET/POST/DELETE /api/admin/dias-bloqueados body:{fecha,motivo?,empleadoId}

Tab Definicoes:

PATCH /api/admin/password body:{passwordActual, passwordNueva}
Mostrar negocio.nombre e negocio.email (lidos de localStorage)

== UX / UI ==

- Todos os textos em PORTUGUES DE PORTUGAL
- Mensagens de sucesso/erro desaparecem apos 3-4 segundos automaticamente
- Loading states e skeletons em todos os pedidos a API
- Formularios limpam os seus campos apos submissao com sucesso
- Todos os botoes 'Reservar' da landing page devem navegar para /reservas
- Instalar: npm install framer-motion
- Usar Framer Motion: AnimatePresence (transicoes), whileInView (scroll), whileHover

== FICHEIROS A CRIAR ==

src/lib/api.ts -> apiFetch, tipos, formatarFecha, getProximasDosSemanas
src/lib/adminAuth.ts -> adminFetch, getToken, setToken, clearToken, tipos admin
src/app/reservas/page.tsx -> pagina de reservas com metadata SEO
src/components/Booking/ -> BookingWizard.tsx ('use client', wizard 5 passos)
src/app/admin/setup/page.tsx -> ('use client') primeira configuracao da senha
src/app/admin/page.tsx -> ('use client') login do administrador
src/app/admin/dashboard/page.tsx -> ('use client') dashboard completo com 7 abas

2. Passo 1 — Criar um novo negocio na API

Antes de integrar a API num novo website, precisas de criar um negocio e obter a sua API key unica. Executa este comando no terminal:

```
curl -X POST https://appointments-api-five.vercel.app/api/negocios \
-H "Content-Type: application/json" \
-d '{"nombre": "Nome do Teu Negocio", "email": "teu@email.com"}'
```

Resposta esperada:

```
{
  "mensaje": "Negocio criado com sucesso",
  "negocio": {
    "id": 2,
    "nombre": "Nome do Teu Negocio",
    "email": "teu@email.com",
    "apiKey": "GUARDA_ESTA_KEY"
  }
}
```

Guarda a apiKey — e o identificador unico do teu negocio. Nao a partilhes publicamente.

Erros possiveis:

Codigo	Mensagem	Solucao
400	Nome e email sao obrigatorios	Verifica que o JSON tem os dois campos
409	Ja existe um negocio com esse email	Usa um email diferente

3. Passo 2 — Configurar variáveis de ambiente

Cria o ficheiro **.env.local** na raiz do projeto:

```
# .env.local (Next.js)
NEXT_PUBLIC_API_URL=https://appointments-api-five.vercel.app
NEXT_PUBLIC_API_KEY=a_tua_api_key_aqui
```

Ou para projetos Vite/React:

```
# .env (Vite)
VITE_API_URL=https://appointments-api-five.vercel.app
VITE_API_KEY=a_tua_api_key_aqui
```

Reinicia o servidor de desenvolvimento apos criar o ficheiro (Ctrl+C e npm run dev) para que as variáveis sejam carregadas.

4. Passo 3 — Setup de admin (primeira vez)

Apos o servidor estar a correr, acede a **/admin/setup** para definir a senha de administrador:

```
# URL de desenvolvimento
http://localhost:3000/admin/setup

# URL de producao
https://seudominio.com/admin/setup
```

Ou via API diretamente:

```
curl -X POST https://appointments-api-five.vercel.app/api/admin/setup \
-H "Content-Type: application/json" \
-d '{"apiKey": "a_tua_api_key", "password": "senha_min_8_chars}"'
```

5. Passo 4 — Configurar funcionarios, servicos e horarios

Apos o login em /admin/dashboard, configura o sistema nesta ordem:

Ordem	Aba	O que fazer
1o	Funcionarios	Adicionar os especialistas com nome, email e telefone
2o	Servicos	Para cada funcionario, adicionar os servicos com nome, duracao e preco
3o	Horarios	Definir os dias e horas de trabalho de cada especialista
4o	Pausas	Opcional: definir intervalos de descanso dentro do horario
5o	Dias Bloqueados	Opcional: bloquear ferias, feriados ou dias sem disponibilidade

6. Passo 5 — Deploy em producao (Vercel)

No Vercel, as variaveis de ambiente do ficheiro `.env.local` **nao sao enviadas para o servidor automaticamente**. Tens de as adicionar manualmente.

1	Abre o projeto no Vercel Dashboard
2	Vai a Settings > Environment Variables
3	Adiciona NEXT_PUBLIC_API_URL com o valor <code>https://appointments-api-five.vercel.app</code>
4	Adiciona NEXT_PUBLIC_API_KEY com a tua API key
5	Seleciona os ambientes: Production, Preview e Development
6	Clica em Save e faz um novo deploy (Deployments > Redeploy)
7	Acede a <code>https://seudominio.com/admin/setup</code> e configura a senha

O ficheiro `.env.local` deve estar no `.gitignore` (por defeito ja esta). Nunca commits as variaveis de ambiente para o repositorio.

7. Resolucao de problemas frequentes

Problema	Causa provavel	Solucao
API key invalida (401)	Key errada ou nao carregada	Reinicia o servidor; verifica .env.local
Sem especialistas na lista	Negocio novo sem funcionarios	Adiciona funcionarios no dashboard
Sem dias disponiveis	Sem horarios ou todos bloqueados	Define horarios no dashboard
Login falha sempre	Senha errada ou admin nao configurado	Acede a /admin/setup
Variaveis undefined em producao	Nao adicionadas no Vercel	Vai a Settings > Environment Variables
Email nao chega ao cliente	Problema na API Resend	Verifica o email do cliente; pode estar em spam
Sessao expira sozinha	JWT dura 24 horas	Faz login novamente